sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Social Sustainability in Software Development

People, Processes, and Values in Sustainable Software Engineering



Gemma Catolino | 2026-2027

Sustainable Software Engineering: Social Software Sustainability

Over the past decade, software engineering has increasingly adopted the concept of sustainability, but most of the attention has focused on **environmental** and **technical** aspects—such as energy efficiency, green data centers, and long-term code maintainability.



Sustainable Software Engineering: Social Software Sustainability

Over the past decade, software engineering has increasingly adopted the concept of sustainability, but most of the attention has focused on **environmental** and **technical** aspects—such as energy efficiency, green data centers, and long-term code maintainability.

In contrast, the **social dimension of sustainability**—which concerns fairness, inclusion, equality, human well-being, and community resilience—has remained under-represented in both research and practice.



Sustainable Software Engineering: Social Software Sustainability

Over the past decade, software engineering has increasingly adopted the concept of sustainability, but most of the attention has focused on **environmental** and **technical** aspects—such as energy efficiency, green data centers, and long-term code maintainability.

In contrast, the **social dimension of sustainability**—which concerns fairness, inclusion, equality, human well-being, and community resilience—has remained under-represented in both research and practice.

Yet, every software system inevitably shapes social relations: how people communicate, how they are treated by algorithms, and how work is organized in teams and companies.



Sustainable Software Engineering: Social Software Sustainability



Social sustainability in software engineering is the capacity of software processes, teams, and systems to support long-term human and societal well-being—by fostering inclusion, trust, fairness, and participation among all stakeholders involved in or affected by software development and use.

Social sustainability in SE is **not a static goal** but a **systemic, evolving condition**:

- grounded in *values* (trust, inclusion, fairness),
- supported by *structures* (leadership, readiness, teamwork), and
- realized through *processes* (reflection, communication, learning).

Sustainable Software Engineering: Social Software Sustainability



Social sustainability in software engineering is the capacity of software processes, teams, and systems to support long-term human and societal well-being—by fostering inclusion, trust, fairness, and participation among all stakeholders involved in or affected by software development and use.

The traditional model of sustainability describes **three pillars** – social, environmental, and economic. While useful for categorization, this view is static and does not capture how these aspects interact in practice.

- grounded in *values* (trust, inclusion, fairness),
- supported by *structures* (leadership, readiness, teamwork), and
- realized through *processes* (reflection, communication, learning).

Sustainable Software Engineering: Multi-Dimensional Software Sustainability

In 2015, **Becker et al.** published the *Karlskrona Manifesto for Sustainability Design*, a foundational work that introduced sustainability as a **multi-dimensional concern** in software engineering.

The authors proposed that sustainability is **not limited to the environment**, but includes **five interrelated dimensions** that must all be considered when designing, developing, and maintaining software systems:

Sustainable Software Engineering: Multi-Dimensional Software Sustainability

In 2015, **Becker et al.** published the *Karlskrona Manifesto for Sustainability Design*, a foundational work that introduced sustainability as a **multi-dimensional concern** in software engineering.

The authors proposed that sustainability is **not limited to the environment**, but includes **five interrelated dimensions** that must all be considered when designing, developing, and maintaining software systems:

Environmental Sustainability – minimizing the ecological footprint of software systems, such as energy use, hardware waste, and emissions.

Economic Sustainability – ensuring that systems remain financially viable over time, balancing cost, maintenance, and value generation.

Technical Sustainability – creating software that remains maintainable, adaptable, and resilient to technological change.

Social Sustainability – addressing equity, inclusion, trust, and participation within the communities affected by or involved in software development.

Individual Sustainability – supporting the well-being, ethics, and personal development of the individuals who design, develop, and use software.

Each dimension interacts with the others; strengthening one (e.g., speed and innovation for technical sustainability) can sometimes weaken another (e.g., personal well-being or social equity).

Sustainable Software Engineering: Multi-Dimensional Software Sustainability

McGuire et al. build directly on Becker's framework but **extend** it with the concept of **stratification**, drawn from systems theory.

They argue that sustainability exists **across multiple interacting strata**, rather than only as parallel dimensions.

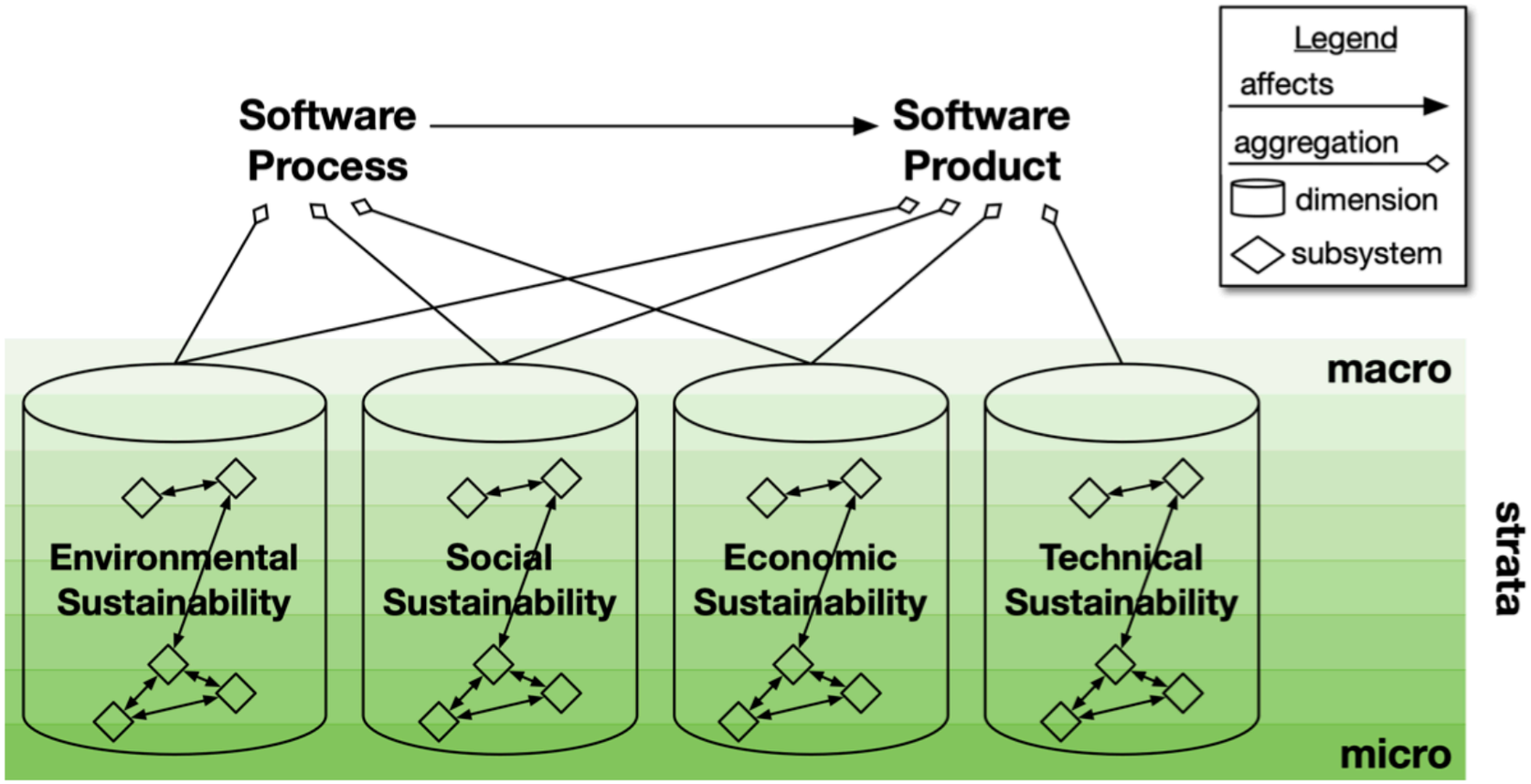
"The model does not include an 'individual' pillar because the individual is just the first strata of the social pillar … Sustainability has different meanings at different strata, from micro to macro."

McGuire thus *removes* the *"Individual"* pillar and reframes it as part of the **social stratum** (the smallest scale within it).

The focus shifts from **dimensions (pillars)** to **levels (strata).** Each stratum constrains and enables the others: an improvement in one can create unsustainability in another.

Hence, McGuire's model adds a **dynamic, relational perspective**—it shows *how the five dimensions interact across levels.*

Sustainable Software Engineering: Multi-Dimensional Software Sustainability



Sustainability is multi-layered (stratified):
It has different meanings at different levels — from individuals and teams to organizations and society.

Process ≠ Product:
Sustainable development practices don't guarantee sustainable software products, and vice versa.

Four key dimensions:
Environmental, Social, Economic, and Technical (technical applies only to the product).

Multisystemic nature:
Sustainability emerges from interactions among social, technical, and ecological systems — not from any single pillar.

Participatory view:
Achieving sustainability requires involvement from diverse stakeholders across systems and levels.

Sustainable Software Engineering: Multi-Dimensional Software Sustainability

McGuire et al. (2023) describe sustainability as **stratified**, meaning that it manifests differently at various **strata** or **levels of abstraction**. The authors explicitly distinguish **micro-level** concerns—such as the well-being of individual people or components—from **macro-level** concerns—such as global environmental or societal systems.

Between these extremes lie intermediate strata that include communities, organizations, or industries, though the paper does not label them with a specific term.

Sustainable Software Engineering: Multi-Dimensional Software Sustainability

McGuire et al. (2023) describe sustainability as **stratified**, meaning that it manifests differently at various **strata** or **levels of abstraction**. The authors explicitly distinguish **micro-level** concerns—such as the well-being of individual people or components—from **macro-level** concerns—such as global environmental or societal systems.

Between these extremes lie intermediate strata that include communities, organizations, or industries, though the paper does not label them with a specific term.

Sustainability, therefore, has **different meanings at different strata**, and an activity that seems sustainable at one layer (for example, within an organization) might be unsustainable at another (for example, for society at large).

Sustainable Software Engineering: Multi-Dimensional Software Sustainability

McGuire et al. (2023) describe sustainability as **stratified**, meaning that it manifests differently at various **strata** or **levels of abstraction**. The authors explicitly distinguish **micro-level** concerns—such as the well-being of individual people or components—from **macro-level** concerns—such as global environmental or societal systems.

Between these extremes lie intermediate strata that include communities, organizations, or industries, though the paper does not label them with a specific term.

Sustainability, therefore, has **different meanings at different strata**, and an activity that seems sustainable at one layer (for example, within an organization) might be unsustainable at another (for example, for society at large).

A project that appears sustainable at one level (for example, a highly efficient development process) may be unsustainable at another (for example, if that efficiency depends on developer burnout or social exclusion).

Therefore, re-examining sustainability in SE means recognizing these **interconnected social layers** and developing methods that address not only the technical and environmental dimensions but also the **human and societal consequences** of our work.

Sustainable Software Engineering:

Aspect	Becker et al. (2015)	McGuire et al. (2023)
Conceptual focus	Dimensions of sustainability (what)	Strata or levels of sustainability (where/how)
Structure	Five independent pillars (Environmental, Economic, Technical, Social, Individual)	Interacting strata forming a micro→macro continuum
View of the individual	Separate “Individual” pillar	Individual = lowest stratum of the social dimension
Interaction between elements	Recognized but not structurally modeled	Explicitly modeled as cross-stratum dependencies
Goal	Awareness of multiple sustainability dimensions	Theoretical model explaining conflicts and dependencies across system levels
Core metaphor	“Pillars” (static categories)	“Layers / strata” (dynamic relationships)

Sustainable Software Engineering: State of the art of Social Software Sustainability

Souza et al. in 2024 conducted a systematic literature review examined over 100 studies to identify how *social sustainability* is conceptualized and applied in software engineering.

The authors found that most research still treats social sustainability **implicitly**, often under labels such as *ethics*, *trust*, *well-being*, or *user inclusion*.

Sustainable Software Engineering: State of the art of Social Software Sustainability

Souza et al. in 2024 conducted a systematic literature review examined over 100 studies to identify how *social sustainability* is conceptualized and applied in software engineering.

The authors found that most research still treats social sustainability **implicitly**, often under labels such as *ethics*, *trust*, *well-being*, or *user inclusion*.

They identified **15 recurring social aspects**, organized into three thematic groups:

- **Human Values:** trust, fairness, equality, privacy, freedom, and respect.
- **Workplace & Team Factors:** collaboration, communication, diversity, transparency, psychological safety.
- **Societal & Ethical Aspects:** inclusion, justice, digital rights, and human development.

Sustainable Software Engineering: State of the art of Social Software Sustainability

Souza et al. in 2024 conducted a systematic literature review examined over 100 studies to identify how *social sustainability* is conceptualized and applied in software engineering.

The authors found that most research still treats social sustainability **implicitly**, often under labels such as *ethics*, *trust*, *well-being*, or *user inclusion*.

They identified **15 recurring social aspects**, organized into three thematic groups:

- **Human Values:** trust, fairness, equality, privacy, freedom, and respect.
- **Workplace & Team Factors:** collaboration, communication, diversity, transparency, psychological safety.
- **Societal & Ethical Aspects:** inclusion, justice, digital rights, and human development.

The study concluded that software engineering needs more **explicit frameworks and measurement approaches** to operationalize these social aspects in practice.

Takeaway: Social sustainability is well recognized conceptually, but underdeveloped methodologically and empirically in SE research.

Sustainable Software Engineering: State of the art of Social Software Sustainability

Ahmad & Gustavsson in 2024 explores how **social sustainability** emerges and is maintained within **large-scale Agile (LSA)** organizations, where multiple Agile teams collaborate within complex corporate environments.

The authors conducted **qualitative case studies** in several large Swedish software companies using interviews, observations, and document analysis. Their goal was to understand which social factors enable long-term, healthy, and productive Agile collaboration.

They identified **eight interrelated factors** that collectively support social sustainability in LSA contexts.

Sustainable Software Engineering: State of the art of Social Software Sustainability

Trust and Relationships
Fostering mutual respect and reliability among distributed teams.

Communication
Maintaining open, transparent, and continuous information flow across teams and hierarchies.

Learning Culture
Encouraging continuous improvement, feedback, and psychological openness to failure.

Self-Organization
Empowering teams to make decisions and manage their own processes.

Decision-Making
Involving people at multiple levels and promoting participatory governance.

Collaboration
Creating shared goals and removing competition between teams.

Leadership and Stress Reduction
Emphasizing servant leadership and avoiding burnout through workload balance.

Psychological Safety
Enabling everyone to speak up without fear of blame or punishment.

Sustainable Software Engineering: State of the art of Social Software Sustainability

These factors form an interconnected **network of relationships**, not isolated dimensions. According to Ahmad and Gustavsson, sustainable large-scale Agile depends on maintaining **balance** among these factors—when one weakens (e.g., trust or communication), the entire social ecosystem suffers.

The authors conclude that **social sustainability is a dynamic capability** in Agile organizations: it must be continuously cultivated through reflection, learning, and leadership attention, rather than achieved once and for all.

Takeaway: Large-scale Agile success depends not only on methods and tools but also on *how people experience collaboration, safety, and shared purpose over time*.

Sustainable Software Engineering: State of the art of Social Software Sustainability

Later, Souza et al. in 2025 tested the Organizational Readiness for Change (ORC) — the shared psychological and behavioral state where members of an organization are committed to implementing a change and confident in their collective ability to do so —theory in the context of social sustainability.

Researchers conducted workshops with software teams to assess two key constructs:

- **Change Commitment (CC):** the belief that social sustainability is important and valuable.
- **Change Efficacy (CE):** the team's confidence in having the resources and skills to implement sustainable practices.

Findings showed that while CC was high, CE was moderate, meaning teams want to act but often don't know how or lack institutional support.

Takeaway: Barriers included short-term delivery focus, lack of leadership engagement, and absence of clear metrics. The authors conclude that organizational culture is a decisive factor for embedding social sustainability into routine software processes.

Sustainable Software Engineering: Barriers and Facilitators

Barriers (hindering factors):

1. **Low Awareness and Conceptual Clarity** – Many practitioners lacked a shared understanding of what "social sustainability" means in the context of software engineering.
2. **Lack of Time and Resources** – Tight project deadlines and delivery pressures left little room for reflective or participatory practices.
3. **Limited Leadership Engagement** – Senior management often supports sustainability in principle but does not provide concrete guidance, budget, or follow-up.
4. **Short-Term Performance Metrics** – Organizational success is measured by speed and output, not by inclusion, well-being, or collaboration quality.
5. **Absence of Evaluation Metrics** – Teams struggle to quantify social sustainability outcomes, making it harder to justify related initiatives.

Sustainable Software Engineering: Barriers and Facilitators

Barriers (hindering factors):

1. **Low Awareness and Conceptual Clarity** – Many practitioners lacked a shared understanding of what "social sustainability" means in the context of software engineering.
2. **Lack of Time and Resources** – Tight project deadlines and delivery pressures left little room for reflective or participatory practices.
3. **Limited Leadership Engagement** – Senior management often supports sustainability in principle but does not provide concrete guidance, budget, or follow-up.
4. **Short-Term Performance Metrics** – Organizational success is measured by speed and output, not by inclusion, well-being, or collaboration quality.
5. **Absence of Evaluation Metrics** – Teams struggle to quantify social sustainability outcomes, making it harder to justify related initiatives.

Facilitators (enabling factors):

1. **Leadership Support and Vision** – When leaders communicate clear sustainability goals and model inclusive behavior, teams are more likely to act.
2. **Training and Capacity Building** – Awareness sessions, reflection workshops (like SusAF), and team learning improve confidence (CE).
3. **Participatory Culture** – Empowering employees to suggest, test, and evaluate sustainability initiatives increases ownership and commitment.
4. **Collaborative Tools and Open Communication** – Transparent communication channels strengthen trust and inclusion.
5. **Alignment with Organizational Values** – When social sustainability is linked to existing company values (e.g., diversity, respect, innovation), it becomes part of normal practice.

Sustainable Software Engineering: The Sustainability Awareness Framework (SusAF)

The **Sustainability Awareness Framework (SusAF)** was developed by **Becker et al.** and later expanded by **Krogstie et al. (2024)**.

It was designed to help software development teams **reflect on sustainability issues during the design and implementation phases**.

Rather than a measurement tool, it is a **facilitation framework** — it creates structured conversations to *raise awareness* of how software decisions affect sustainability across multiple dimensions.

Sustainable Software Engineering: The Sustainability Awareness Framework (SusAF)

The **Sustainability Awareness Framework (SusAF)** was developed by **Becker et al.** and later expanded by **Krogstie et al. (2024)**.

It was designed to help software development teams **reflect on sustainability issues during the design and implementation phases**.

Rather than a measurement tool, it is a **facilitation framework** — it creates structured conversations to *raise awareness* of how software decisions affect sustainability across multiple dimensions.

SusAF is grounded in two key ideas:

- The **Karlskrona Manifesto** (Becker et al., 2015), which defines sustainability as a *multi-dimensional concern* (environmental, economic, technical, individual, and social).
- The principle that **software has indirect and long-term effects**—it influences people, organizations, and society well beyond immediate functionality.

Krogstie et al. (2024) describe SusAF as a *"design-oriented reflective tool"* that prompts developers to think about both the *immediate* and *systemic* consequences of their design choices.

Sustainable Software Engineering: The Sustainability Awareness Framework (SusAF)

Dimension	Description	Example Questions in a SusAF Workshop
Environmental	Ecological impact, energy use, and resource consumption.	"Does our software reduce or increase energy demand?"
Economic	Long-term financial viability, cost efficiency, and value distribution.	"Is this solution economically sustainable for all stakeholders?"
Technical	Maintainability, resilience, and scalability of systems.	"Will our system still be maintainable in 5-10 years?"
Social	Human relationships, inclusion, fairness, and participation.	"How does our product affect users' trust, inclusion, or well-being?"
Individual	Health, autonomy, and growth of individuals involved.	"Are we supporting developers' and users' well-being and autonomy?"

Sustainable Software Engineering: The Sustainability Awareness Framework (SusAF)

Dimension	Description	Example Questions in a SusAF Workshop
Environmental	Ecological impact, energy use, and resource consumption.	"Does our software reduce or increase energy demand?"
Economic	Long-term financial viability, cost efficiency, and value distribution.	"Is this solution economically sustainable for all stakeholders?"
Technical	Maintainability, resilience, and scalability of systems.	"Will our system still be maintainable in 5–10 years?"
Social	Human relationships, inclusion, fairness, and participation.	"How does our product affect users' trust, inclusion, or well-being?"
Individual	Health, autonomy, and growth of individuals involved.	"Are we supporting developers' and users' well-being and autonomy?"

SusAF also introduces **three time horizons**, showing how sustainability effects unfold over time:

- **Immediate effects:** Short-term consequences (e.g., user stress due to bad UX).
- **Enabling effects:** Medium-term conditions that enable or constrain sustainability (e.g., documentation, training, user empowerment).
- **Structural effects:** Long-term systemic outcomes (e.g., shaping work culture, digital inequality, or dependency on infrastructure).

This *temporal layering* helps participants see sustainability as **dynamic and long-term**, not only as short-term efficiency.

Sustainable Software Engineering: The Sustainability Awareness Framework (SusAF)

SusAF is typically used in **facilitated workshops** (2–4 hours).

Team members (developers, designers, managers, sometimes users) are invited to **map their system or project** across the five dimensions and three time horizons.

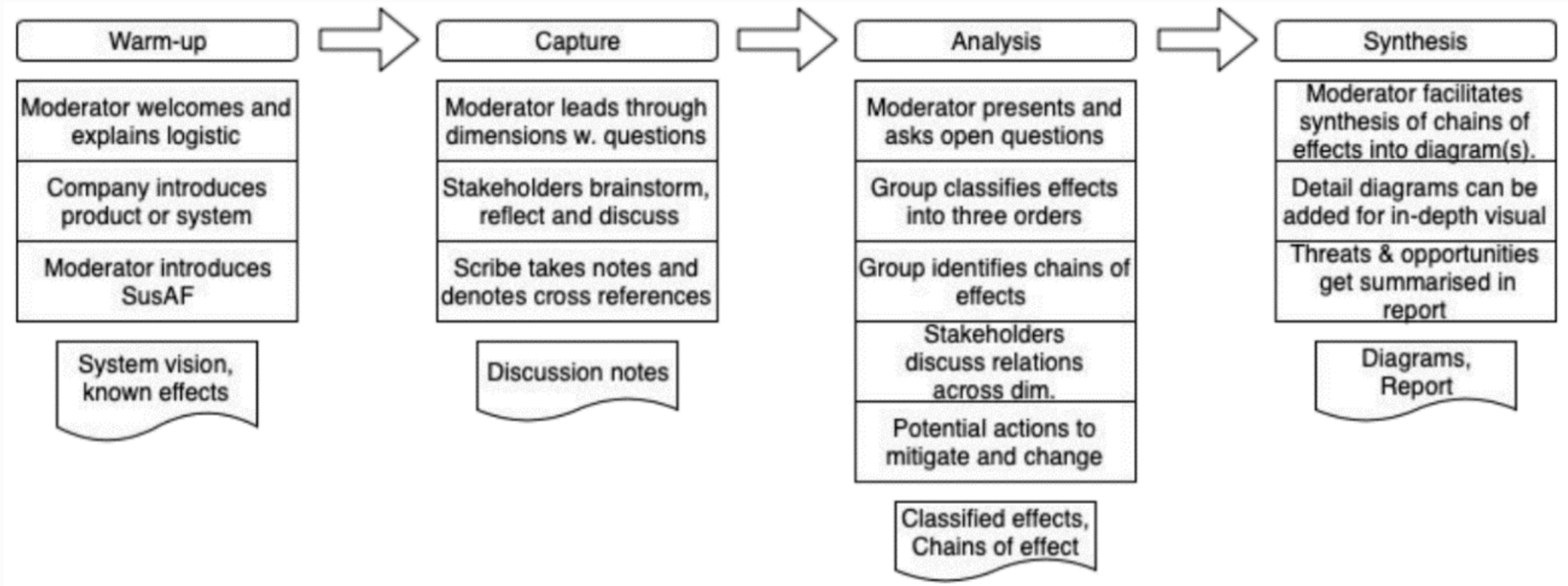
The outcome is often a **visual map** or **poster**, highlighting trade-offs, neglected dimensions, and potential improvements.

Sustainable Software Engineering: The Sustainability Awareness Framework (SusAF)

SusAF is typically used in **facilitated workshops** (2-4 hours).

Team members (developers, designers, managers, sometimes users) are invited to **map their system or project** across the five dimensions and three time horizons.

The outcome is often a **visual map** or **poster**, highlighting trade-offs, neglected dimensions, and potential improvements.



Sustainable Software Engineering: The Sustainability Awareness Framework (SusAF)

Krogstie et al. (2024) report that teams using SusAF gained:

- higher awareness of social and human impacts,
- deeper cross-role discussions, and
- identification of design blind spots.

Based on workshops with software teams in Norway and other European contexts.

The **social dimension** was discussed most frequently, especially topics like *trust*, *inclusion*, *diversity*, *and collaboration*.

However, participants found this dimension **the hardest to evaluate** — unlike energy consumption or performance metrics, social outcomes are qualitative and long-term.

Teams often discovered new perspectives — for example, realizing how usability choices affected accessibility or how deadlines impacted developer well-being.

Takeways: SusAF is **effective for reflection and dialogue**, but requires **organizational commitment** to translate awareness into concrete actions (ties directly to Souza et al. 2025's ORC theory).

Sustainable Software Engineering: Is Your Software Socially Sustainable? Find Out!

Equitable Access – Software should be usable and accessible to everyone, regardless of income, ability, language, or device.

https://blogs.dal.ca/openthink/is-your-software-socially-sustainable-heres-how-to-find-out/

Sustainable Software Engineering: Is Your Software Socially Sustainable? Find Out!

Equitable Access – Software should be usable and accessible to everyone, regardless of income, ability, language, or device.

Inclusivity & Representation – Systems must reflect diverse user needs and perspectives, avoiding stereotypes or exclusion.

https://blogs.dal.ca/openthink/is-your-software-socially-sustainable-heres-how-to-find-out/

Sustainable Software Engineering: Is Your Software Socially Sustainable? Find Out!

Equitable Access – Software should be usable and accessible to everyone, regardless of income, ability, language, or device.

Inclusivity & Representation – Systems must reflect diverse user needs and perspectives, avoiding stereotypes or exclusion.

User Well-Being – Software should enhance users' quality of life, not exploit attention, induce stress, or harm mental health.

https://blogs.dal.ca/openthink/is-your-software-socially-sustainable-heres-how-to-find-out/

Sustainable Software Engineering: Is Your Software Socially Sustainable? Find Out!

Equitable Access – Software should be usable and accessible to everyone, regardless of income, ability, language, or device.

Inclusivity & Representation – Systems must reflect diverse user needs and perspectives, avoiding stereotypes or exclusion.

User Well-Being – Software should enhance users' quality of life, not exploit attention, induce stress, or harm mental health.

Community Engagement – Strengthens social connections and empowers communities through participation and collaboration.

https://blogs.dal.ca/openthink/is-your-software-socially-sustainable-heres-how-to-find-out/

Sustainable Software Engineering: Is Your Software Socially Sustainable? Find Out!

Equitable Access – Software should be usable and accessible to everyone, regardless of income, ability, language, or device.

Inclusivity & Representation – Systems must reflect diverse user needs and perspectives, avoiding stereotypes or exclusion.

User Well-Being – Software should enhance users' quality of life, not exploit attention, induce stress, or harm mental health.

Community Engagement – Strengthens social connections and empowers communities through participation and collaboration.

Ethical Data Practices & Trust – Data handling must be transparent, secure, privacy respectful, promoting long-term trust.

https://blogs.dal.ca/openthink/is-your-software-socially-sustainable-heres-how-to-find-out/

Sustainable Software Engineering: References

McGuire, R., Becker, C., Penzenstadler, B., & Dick, M. (2023). *Sustainability is Stratified: Toward a Better Theory of Sustainable Software Engineering*. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023)*. IEEE/ACM.

Souza, A. C. M. de, Silva, A. S., Gomes, J. M., & Pinto, G. (2025). *Promoting Social Sustainability within Software Development through the Lens of Organizational Readiness for Change Theory. Information and Software Technology*, 184, 107755.

Souza, A. C. M. de, Santos, R., Silva, A. S., & Pinto, G. (2024). *Social Sustainability Approaches for Software Development: A Systematic Literature Review*. In *Product-Focused Software Process Improvement (PROFES 2023)*, Lecture Notes in Computer Science, vol. 14483. Springer.

De Souza, A. C. M. (2023). *Social Sustainability Approaches for a Sustainable Software Product. ACM SIGSOFT Software Engineering Notes*, 48(1), 1–8.

Krogstie, J., Chasanidou, D., & Krogstie, B. (2024). *Social Sustainability by Design: Experiences with the Sustainability Awareness Framework (SusAF)*. In *Proceedings of the REFSQ 2024 Workshops*.

Ahmad, M., & Gustavsson, T. (2024). *Social Sustainability and Large-Scale Agile Software Development*. In *Proceedings of the 32nd International Conference on Information Systems Development (ISD 2024)*, Gdańsk, Poland.

Becker, C., Chitchyan, R., Duboc, L., Easterbrook, S., Penzenstadler, B., Seyff, N., & Venters, C. C. (2015). *The Karlskrona Manifesto for Sustainability Design. Proceedings of the 37th International Conference on Software Engineering (ICSE 2015)*, 231–232.

https://sustainabilitydesign.org/karlskrona-manifesto/

https://blogs.dal.ca/openthink/is-your-software-socially-sustainable-heres-how-to-find-out/

sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Social Sustainability in Software Development

People, Processes, and Values in Sustainable Software Engineering



Gemma Catolino | 2026-2027